

朱松

男 | 年龄：22岁 | 18832179774 | zhusong262@gmail.com

求职意向：Agent开发、机器学习、云原生 | 期望城市：北京



个人优势

- 独立项目落地能力：独立完成从需求分析 → 架构设计 → 代码实现 → 容器化部署的全流程，具备单兵作战能力，能够快速响应业务需求并交付可用产品。
- AI 技术栈深度实践：在单一项目中深度实践了 LangGraph Agent 编排、双层 RAG 检索、知识图谱构建、Prompt 工程等核心技术，技术理解不浮于表面。
- 工程化意识：注重系统健壮性，在项目中主动设计了五层容错保护链、人工审核网关等生产级方案，具备将 AI 能力转化为可靠产品的思维。
- 快速学习与技术敏感度：持续跟踪 LangGraph、GraphRAG 等前沿框架，能够在短时间内理解新技术原理并落地到实际项目中。

专业技能

- AI 大模型与智能体开发：熟练掌握 LangGraph、LangChain 等智能体框架，深刻理解 ReAct 架构、图状态机（State Graph）流转、LLM Tool-Calling 及 RAG 原理。具备从零构建高可靠 Agent 调度引擎的实战经验；能够快速对接并深度集成主流大模型（如 Google Gemini、通义千问、智谱 GLM 等）API。
- RAG 与知识图谱：掌握 ChromaDB、FAISS 等向量数据库原理与使用，熟悉 Neo4j 知识图谱构建与 Cypher 查询；具备向量检索 + 图谱检索双层混合检索系统的设计与实现经验。
- Prompt Engineering：熟悉系统提示词分层设计、Few-shot/CoT 提示策略、长对话上下文压缩等技术；具备生产级 Prompt 工程实践经验。
- 编程语言与框架：熟练掌握 Python，熟悉 Go 语言；能够基于 FastAPI 进行后端服务开发，掌握 Goroutine、Channel 等并发机制。
- 工程化与云原生部署：熟练掌握 Docker 容器化技术与 Kubernetes (K8s) 集群部署；能够独立完成应用镜像构建、Dockerfile 编写，以及 K8s 环境下的微服务弹性部署与服务治理；熟悉 CI/CD 自动化流程，具备 Linux 环境下的开发与运维能力。

项目经历

GraphFlow：基于 LangGraph 的企业智能办公多 Agent 系统 架构设计与核心研发 2026.03-至今

内容：

项目链接：<https://github.com/Zhu-song/langgraph-multi-agent>

核心技术：

- 语言与框架：Python, FastAPI, SSE (Server-Sent Events)
- 大模型与框架：LangGraph (ReAct 架构)、LangChain、Prompt Engineering
- 检索与索引：ChromaDB 向量检索、Neo4j 知识图谱、LightRAG 双层混合检索 (local/global/hybrid)
- 基础设施：SQLite 状态持久化、Docker Compose、Human-in-the-Loop 审核机制

项目背景：

● 针对传统 LLM Agent 在工具扩展性差、RAG 检索在跨文档关联推理中的信息断裂、以及高危工具缺乏安全管控等问题，设计并实现了一套基于图状态机（State Graph）的高可靠智能体调度系统。系统通过引入子图解耦、基于 interrupt 机制的人工审核网关以及反思机制（Reflection），实现了从单线工具调用到多工具并发路由的深度演进，并集成向量检索与知识图谱双层混合检索引擎，有效提升了复杂意图下的系统健壮性与可观测性。

业绩:

项目经历:

- 基于图状态机的 ReAct 推理引擎:
基于 LangGraph StateGraph 构建 ReAct 推理循环, 通过 llm.bind_tools() 将 15 种工具绑定至 LLM 实现自主决策调用; 引入子图 (Subgraph) 解耦工具执行与人工审批逻辑, 利用 Send API 实现多工具并发路由, 显著提升了复杂意图下的响应效率。
- 双层混合检索引擎:
集成 ChromaDB 向量检索与 Neo4j 知识图谱, 实现 local/global/hybrid 三种检索模式; 基于 LightRAG 框架开发 Agent 路由模块, 根据查询复杂度动态分配检索策略, 并引入 MD5 去重的增量文档更新机制, 解决了知识库规模化后的维护效率问题。
- Human-in-the-Loop 安全审核网关:
基于 LangGraph interrupt 机制设计高危工具拦截 workflow, 实现状态持久化与暂停/恢复; 配合五层装饰器保护链 (全局兜底→自动重试→三级降级→链路日志→会话限流), 保障系统在高并发场景下的健壮性与可观测性。
- 反思机制与 Prompt 工程:
设计 5 套分层系统提示词, 覆盖调度决策、全局约束、RAG 检索与问答等环节; 引入 Reflection 自省纠错工具, 对 Agent 输出进行事实一致性二次校验, 有效降低了事实幻觉率。

项目业绩:

- 检索增强: 引入向量+知识图谱双层混合检索, 复杂关联问题的召回率提升 30%。
- 质量调优: 引入 Reflection 自省机制与分层 Prompt 工程, Agent 事实幻觉率降低 35%。
- 工程可靠: 五层容错保护链 + interrupt 审核机制, 系统异常恢复时间缩短至 秒级。
- 开发提效: 子图解耦 + 工具标准化注册机制, 新工具集成周期由 1天缩短至 2小时。

实际业务落地应用:

- 企业知识库智能问答: 依托双层检索整合企业内部文档, 支持权限查询与流程解读, 高效解决资料查阅难题, 减少行政咨询压力。
- 多系统协同办公助手: 快速对接各类办公系统, 自动处理流程发起、事务统筹等工作, 减少重复操作, 提升办公效率。
- 办公操作安全管控: 通过人工审核拦截高危办公操作, 实现权限管控与操作留痕, 保障企业内部信息安全。

办公场景业务价值:

- 办公效率显著提升, 重复性事务自动化率达 70%, 员工事务性耗时减少 40%
- 内部知识高效流转, 新人熟悉业务流程周期缩短 30%
- 全程合规可控, 有效规避内部数据泄露与违规操作风险

基于知识图谱与向量双引擎的智能推理 Agent

核心开发者

2026.02-2026.04

内容:

项目链接: <https://github.com/Zhu-song/RAG>

核心技术:

- 语言与框架: Python , FastAPI , MCP 协议(Model Context Protocol)
- 大模型与框架: 智谱GLM-4、LangGraph (多智能体框架)、Prompt Engineering
- 检索与索引: GraphRAG , FAISS , BM25 混合检索, Reranker (BGE-Reranker)
- 基础与评估: Neo4j , Docker , RAGAS

项目背景:

- 针对传统RAG在处理非结构化长文本时面临的“语义孤岛”与复杂关联推理问题, 设计并实现了一套集成知识图谱 (KG) 与向量索引 (Vector Index) 的双引擎检索增强系统。系统通过Agent实现了从局部语义定位到全局拓扑推理的深度演进, 有效解决了跨文档多跳推理 (Multi-hop Reasoning) 中的信息断裂问题。

业绩:

项目经历:

- 多源异构数据建模与图构建
利用Python开发自动化ETL模型, 利用 Recursive Chunking 策略优化长文本切片。调用GLM-4提出实体与三元组, 并利用Neo4j

构建知识图谱；针对图谱规模爆炸问题，引入实体消歧与规范化算法，保证图谱结构的逻辑一致性。

●混合设计

实现了一套三路混合搜索方案（向量+关键词+图）。开发Agent路由模块，根据用户意图识别（意图分类）动态分配搜索权重：简单查询调用FAISS与BM25，复杂关联查询通过图算法进行多路径跳搜优，显著提升了上下文的信息密度。

●基于 MCP 协议的插件化工具集集成

首先落地MCP (Model Context Protocol)协议，将 Agent 与外部工具（如飞书实时消息、GitHub 代码库、本地数据库）进行标准化连接。通过统一的服务器接口规范，解决了多智能体在不同环境下的上下文了对接问题，彻底降低了新功能的集成成本。

●多智能体协同与自反思 workflow

基于 LangGraph 构建“规划-执行-反思”闭环。引入 Critic Agent 进行事实一致性校验，将复杂任务的输出严谨性显著提升。

项目业绩：

- 检索突破：引入 GraphRAG 实现多路混合检索，复杂问题的召回率提升 35%。
- 质量调优：基于 RAGAS 框架闭环优化，使 Agent 事实幻觉率降低 40%。
- 重排优化：集成 BGE-Reranker 执行二次精排，使上下文精准度提升 20%。
- 工程提效：率先落地 MCP 协议，将工具集成周期由 2 天缩短至 2 小时。

实际业务落地应用：

依托双引擎检索与插件化 Agent 架构，落地研发故障诊断与排障助手场景：

- 故障知识图谱构建：对接项目代码库、API 文档、故障日志、历史工单，构建服务依赖、错误类型、解决方案的关联图谱，实现故障根因的多跳关联推理。
- 研发场景智能问答：支持开发人员用自然语言查询报错原因、接口说明、历史解决方案，跨代码、文档、日志多源检索，解决排障依赖个人经验的痛点。
- 插件化工具集成：基于 MCP 协议对接 GitLab、日志平台、CI/CD 系统，实现故障信息一键拉取、代码片段定位、修复建议推送，打通诊断与修复闭环。

业务价值体现：

- 排障效率：典型线上故障平均解决时间（MTTR）缩短 50%，开发人员排障耗时减少 45%。
- 知识沉淀：故障处理经验 100% 结构化沉淀，新人开发上手周期缩短 40%。
- 快速适配：插件化架构可快速迁移至运维监控、系统审计等技术支撑场景，具备广泛落地潜力。

企业级大语言模型（LLM）与飞书智能助手集成落地

独立开发

2026.02-2026.02

内容：

● 项目背景：为降低团队获取信息的并提升办公与研发效率，需要从零到一开发的早期成本集成在飞书内部的AI机器人，提供高质量的多轮对话、文本处理与辅助编程等定制化服务。

● 核心贡献：

多模型统一接入与智能路由：基于Python/Go构建网关服务，深度对接Google Gemini、通义千问（Qwen）、智谱等全部主流大模型API。设计了统一的接口封装与网关层，支持根据不同指令或类型灵活切换底层大模型。

飞书开放平台深度集成：负责飞书企业级应用的全生命周期配置。处理复杂的API鉴权（租户访问令牌），配置Webhook事件订阅，实现飞书服务端与内部AI服务的安全、稳定通信的机制。

流式交互与上下文管理设计：针对大模型生成运行的痛点，打通大模型流输出式（SSE）到飞书消息流的平滑透传，大幅减少用户感知的首字响应延迟；基于缓存的会话状态管理设计，支持历史上下文记忆与多轮连贯对话。

服务容器化部署：编写 Dockerfile 将业务代码容器化，并凭借已有的自动化部署经验，保障了应用在不同环境（Dev/Prod）下的平滑迁移与快速迭代。

业绩：

- 迄今为止成功将外部多模态大语言模型无缝内嵌至企业办公流，系统稳定运行。
- 日均稳定处理多轮对话及文本任务请求150+次，外汇为团队节省日常知识搜索与代码查错工时约30%。
- 深度优化交互体验，打通流式输出（SSE）队列，将平均首字响应延迟（TTFT）严格控制在1.5秒内，大幅提升了终端的用户体验。

内容:

项目链接: <https://github.com/Zhu-song/k8s-microservice-platform>

● 项目背景

需要将传统的单体python应用迁移到微服务架构，并实现自动化部署、弹性扩缩容和实时监控。目标是提高系统可用性、降低运维成本，并支持快速迭代发布。

● Kubernetes集群搭建与优化

使用kubeadm部署生产级Kubernetes集群（1 Master + 3 Worker Nodes），配置高可用ETCD和负载均衡。

优化集群性能：通过kubelet资源限制（CPU/Memory Requests/Limits）、启用PodDisruptionBudget保障服务稳定性。实现Calico网络插件集成，解决跨节点Pod通信问题。

● 微服务容器化与部署

将Spring Boot微服务打包为Docker镜像，编写Dockerfile并推送至私有镜像仓库（Harbor）。

使用Helm Chart统一管理微服务部署模板，实现环境隔离（Dev/Test/Prod）。

通过Kustomize实现不同环境的差异化配置。

● 自动化CI/CD流水线

设计Jenkins Pipeline，实现代码提交后自动构建镜像、运行单元测试、推送镜像仓库，并触发kubectl rollout更新Pod。

集成GitLab Webhook，实现代码合并到主分支时自动部署到测试环境。

● 监控与日志系统

部署Prometheus+Grafana监控集群资源（Node/Pod CPU/Memory利用率）和微服务健康状态。

配置自定义告警规则（如Pod CrashLoopBackOff、节点资源超限）。

使用EFK（Elasticsearch + Fluentd + Kibana）收集容器日志，实现故障快速定位。

业绩:

● 成功部署10+微服务，发布周期从2周缩短至1天，资源利用率提升40%。

● 集群稳定性达到99.95%，故障恢复时间从30分钟降低至5分钟以内。

● 通过监控告警系统提前发现并解决3起潜在生产事故，减少业务损失。

教育经历

河北地质大学华信学院 本科 计算机科学与技术

2025-2027

基础原理扎实，动手操作能力相对较好，善于沟通协作完成项目，容器技术基础扎实，熟悉项目开发步骤。

资格证书

驾驶证C1 普通话二级甲等 HCIP华为认证ICT高级工程师 RHCSA红帽认证系统管理员 ACE阿里云高级工程师认证
计算机二级